

Analysis: Building Palindromes

For each query $[L, R]$, we have to figure out if it is possible to make a palindrome from substring $[L, R]$ of the original block string.

For any substring, we can create all possible permutations of it and check if any of those is a palindrome. For a substring of length l , in the worst case, there would be [Factorial\(\$l\$ \)](#) permutations, and to check if a string of length l is palindrome is $O(l)$. So the cost of each query would be $O(N \times \text{Factorial}(N))$ and thus wouldn't fit within the time limit.

One important observation is that in a palindrome, at most one character can appear odd number of times. If more than one character appears odd number of times in a string, it is impossible to rearrange that string to form a palindrome.

If all characters in a string are present even times, we can just divide the characters into two identical sets. Then we can make two strings with those two sets such that one string is the reverse of the other one. Finally concatenate them to get a palindrome.

If we have only one character let's say x , which is present odd number of times, we can set aside one x . Then we get a set of character where all characters are present even number of times, and we can construct a palindrome in somewhat similar way as described above, this time we can just put the x in between those two strings.

Test set 1 (Visible)

For each query, we can count the frequencies of all characters in the substring and decide if it is possible to make a palindrome or not. The complexity of solving each query in this approach is the length of the substring, which is $O(N)$. Total complexity of this approach is $O(N \times Q)$, which will be sufficient for test set 1.

Test set 2 (Hidden)

We need to count the frequencies of each characters in a query substring, but can we make the counting of frequencies faster?

Notice that, if we calculate prefix sum of frequencies for each character on the whole input string, we can get the frequency of any character in any given substring in $O(1)$ time.

In this approach, the time required to compute the prefix sum of the frequencies for each character is $O(N)$ and hence $O(N \times |\text{character set}|)$ for all. And for each query substring, we can check parity of the frequency for each of the characters in $O(|\text{character set}|)$ time. So for Q queries, the overall complexity of this approach is $O((N + Q) \times |\text{character set}|)$, which is sufficient for test set 2.

Analysis: Diverse Subarray

Consider finding the optimal solution that is a prefix of the input. That is, let us only consider taking ranges of trinkets $[1, r]$ for any r . This reduces the problem to finding prefix sum maximums. For example, consider this sequence of trinkets:

1, 1, 2, 1, 3, 2, 3, 2, 2

Assume that $S=2$.

The first and second occurrence of a type can be modeled as a $+1$ event since we can take the trinkets. The $S+1$ th occurrence can be a $-S$ event, since now we cannot take any trinkets of that type. Finally, any further occurrence of that type can be a $+0$ since we still cannot take them.

Thus, we have the following events:

$+1, +1, +1, -2, +1, +1, +1, -2, +0$

A prefix sum ending at index r in our events corresponds to the score for taking the subarray $[1, r]$. This idea can be generalized to any value of S , since all events until the $S+1$ th can be $+1$ events.

Test set 1 (Visible)

A naive algorithm can be realized using the above strategy. It can be applied to each potential left point (instead of always starting at 1). This explicitly considers every possible range $[l, r]$ and computes the number of trinkets. The events can be calculated by keeping track of how many times each type has occurred while sweeping across the trinkets and keeping a running sum of events. The count of types can be handled with a frequency table. Any map from type to count can implement this, for example, an array or hashtable. The strategy uses $O(N)$ time to process the solution starting at a particular left point. There are $O(N)$ left points. Thus, the time complexity is $O(N^2)$. This is sufficient for Test set 1.

Test set 2 (Hidden)

The previous algorithm will not be fast enough for the Test set 2. Observe that the events for the l th left point are closely related to those for the $l+1$ th left point. In particular, only events corresponding to the type of the trinket at index l change. In fact, only the $-S$ for that type changes. It moves over to the next $+0$ event for that type if it exists. The previous $-S$ event becomes a $+1$. This means that the events only change at a constant number of places.

Using this observation, we can think about this as a data structure problem. We require a data structure that supports two operations:

1. Change the value at an index.
2. What is the maximum prefix sum?

This can be used to maintain our events and find the best solution for each left point respectively. These operations can be achieved using a modified segment tree. Each node in the segment tree can be modified to store the sum of the elements covered by that node. Also, each node should store the maximum sum of any prefix of elements that are covered by that node. Such a tree can support both operations in $O(\log(N))$ time. We can achieve a final complexity of $O(N \log(N))$.

Analysis: Energy Stones

Test set 1 (Visible)

For this test set, it is guaranteed that $S_i = S_j$ for all i, j . For simplicity, we will assume that we never eat a stone with zero energy. Consider two energy stones i and j that will be eaten back-to-back. If $L_i > L_j$ then we should eat i before j . This is because stone i loses energy faster than j , so taking it first will result in a smaller overall loss of energy.

Thus, no matter which set of energy stones are eaten, that set should be eaten in non-increasing value of L_i . So we should first sort the stones by L_i and then the only decision to be made is which stones should be eaten and which should not be eaten. This reduces the problem to a [0/1 Knapsack](#) problem. This can be solved with [dynamic programming](#).

Define $\text{max_energy}(\text{time}, i)$ as the maximum total energy that can be achieved given the current time and considering only the suffix of energy stones sorted in decreasing L_i from i to N . The recurrence relation for this function considers two cases. Either take the i -th energy stone (with its energy adjusted by the time), or do not take it. So, $\text{max_energy}(\text{time}, i)$ is the maximum of:

- $\text{max_energy}(\text{time} + S_i, i+1) + \max(0, E_i - L_i \cdot \text{time})$
- $\text{max_energy}(\text{time}, i+1)$

The maximum possible time is the sum of all S_i because an optimal strategy might eat all the stones and will not use any time waiting. Call this $\text{sum}(S)$. The time complexity of this approach can be described as $O(N \times \text{sum}(S))$. This is fast enough for both test sets. However, sorting energy stones by L_i is incorrect for Test set 2.

Test set 2 (Hidden)

We will need to find a different way to order the energy stones to solve Test set 2. As before, consider two energy stones i and j assuming that we can take both i and j without either going to zero energy. We know that S_i might not equal S_j . However, there is an ordering for taking both i and j that is always optimal. Observe that $S_i L_j$ is the total loss of energy if i is used first. Likewise, $S_j L_i$ is the loss if j is used first. Thus, if $S_i L_j < S_j L_i$ then taking i first leads to a smaller overall loss of energy. It may not be obvious that we should always take i before j even if it leads to a smaller loss of energy. This is because there may be other stones between i and j in some potential ordering. However, if i and j are adjacent in some ordering, then we will achieve more energy by swapping them if $S_i L_j > S_j L_i$. Applying this rule iteratively will eventually sort the stones. Therefore, this rule defines an ordering on our energy stones.

Formally, suppose for a contradiction, we have an optimal solution that eats X stones in the order $P_1, P_2, \dots, P_X$, where each stone gives Duda a positive amount of energy, but there exists an i such that $S_{P_i} L_{P_{i+1}} > S_{P_{i+1}} L_{P_i}$. If we swap the order we eat these two stones, we gain exactly $S_{P_i} L_{P_{i+1}}$ more energy and lose at most $S_{P_{i+1}} L_{P_i}$ (we may lose less than that, if the stone's energy drops to zero).

Since we assumed that $S_{P_i} L_{P_{i+1}} > S_{P_{i+1}} L_{P_i}$, this would increase the amount of energy Duda gains, which contradicts the assumption that this is an optimal solution.

Thus, we can use the dynamic programming solution from Test set 1 to solve Test set 2 with the same time complexity.

The reader may have noticed that this sort order is equivalent to comparing fractions; it is the same as sorting by S_i/L_i . However, one must be careful when $L_i=0$.